HACETTEPE UNIVERSITY

COMPUTER ENGINEERING DEPARTMENT

BBM234 Computer Organization - Spring 2026

SystemVerilog Project: MIPS Single-Cycle Processor

Name/ Surname : Zeynep Tuğçe Tuzsuz

Student No: 2230356073

1. **Please indicate how many hours you spent on this lab.**

   I spent approximately 10 hours for his lab.

2. A completed version of Table 1 and Table 2.

Cycle	reset	pc	instr	branch	srca	srcb	aluout	zero	pcsrc	writedata	memwrite	read data
1	1	00	addi $2, $0, 5 20020005	0	0	5	5	0	0	0	0	x
2	0	04	addi $3, $0, 12 2003000c	0	0	C	C	0	0	0	0	x
3	0	08	addi $7, $3, -9 20067fff7	0	C	-9	3	0	0	0	0	x
4	0	0C	or $4, $7, $2 00e22025	0	3	5	7	0	0	5	0	x
5	0	10	and $5, $3, $4 00642824	0	C	7	4	0	0	7	0	x
6	0	14	add $5, $5, $4 00a42820	0	4	7	B	0	0	7	0	x
7	0	18	beq $5, $7, end 10a7000a	1	B	3	8	0	0	3	0	x
8	0	1C	slt $4, $3, $4 0064202a	0	C	7	0	1	0	7	0	x
9	0	20	beq $4, $0, around 10800001	1	0	0	0	1	1	0	0	x
10	0	28	slt $4, $7, $2 00e2202a	0	3	5	1	0	0	5	0	x
11	0	2C	add $7, $4, $5 00853820	0	1	B	C	0	0	B	0	x
12	0	30	sub $7, $7, $2 00e23822	0	C	5	7	0	0	5	0	x
13	0	34	sw $7, 68($3) ac670044	0	C	44	50	0	0	7	1	x
14	0	38	lw $2, 80($0) 8c020050	0	0	50	50	0	0	0	0	7
15	0	3C	j end 08000011	0	X	X	X	X	0	0	0	x
16	0	44	sw $2, 84($0) ac020054	0	0	54	54	0	0	7	1	x

Table 1. **First sixteen cycles of executing mipstest.asm**

Instruction	$Op_{5:0}$	RegWrite	RegDst	$AluSrc_{1:0}$	Branch	bne	MemWrite	MemtoReg	$ALUOp_{1:0}$	Jump
R-type	000000	1	1	00	0	0	0	0	10	0
lw	100011	1	0	01	0	0	0	1	00	0
sw	101011	0	X	01	0	0	1	X	00	0
beq	000100	0	X	00	1	0	0	X	01	0
addi	001000	1	0	01	0	0	0	0	00	0
j	000010	0	X	XX	0	0	0	X	XX	1
ori	001101	1	0	10	0	0	0	0	11	0
bne	000101	0	X	00	0	1	0	X	01	0

Extended functionality. Main Decoder:

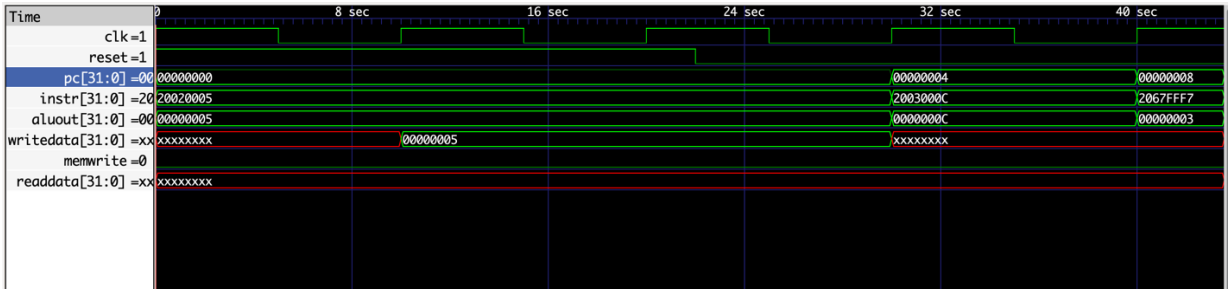
Table 2. **Extended functionality. Main Decoder**

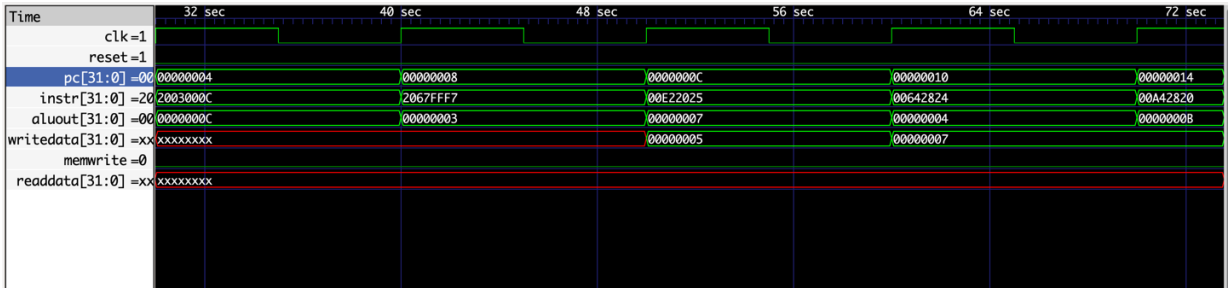
Extended functionality. ALU Decoder:

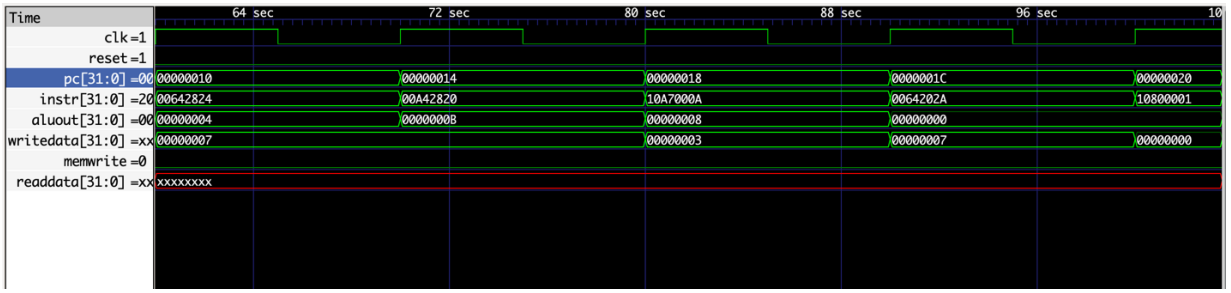
$ALUOp_{1:0}$	Meaning
00	Add
01	Subtract
10	Look at funct field
11	Or

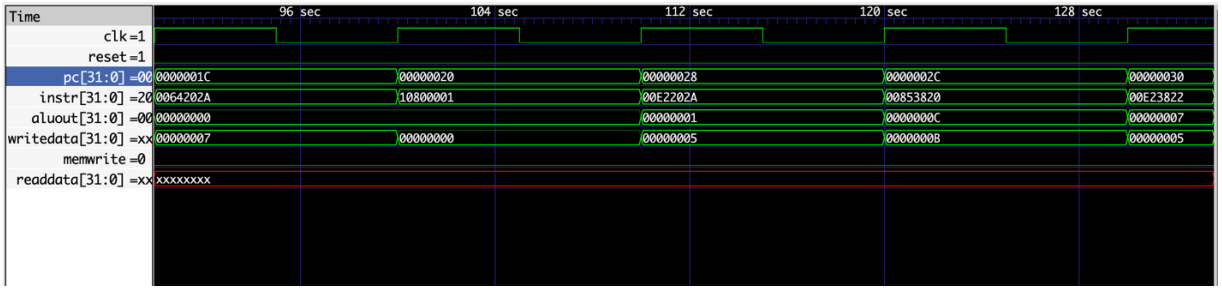
3. An image of simulation waveforms showing correct operation of the processor.

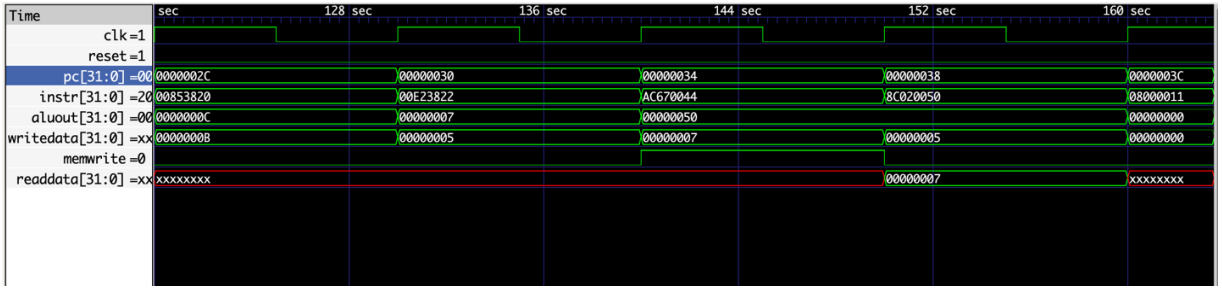
Yes, the processor successfully writes the correct value to address 84. The final sw instruction stores the value 7 into memory address 84.

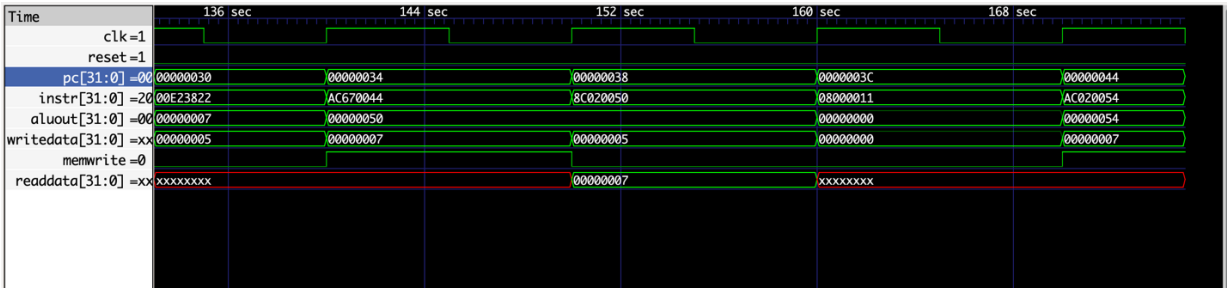




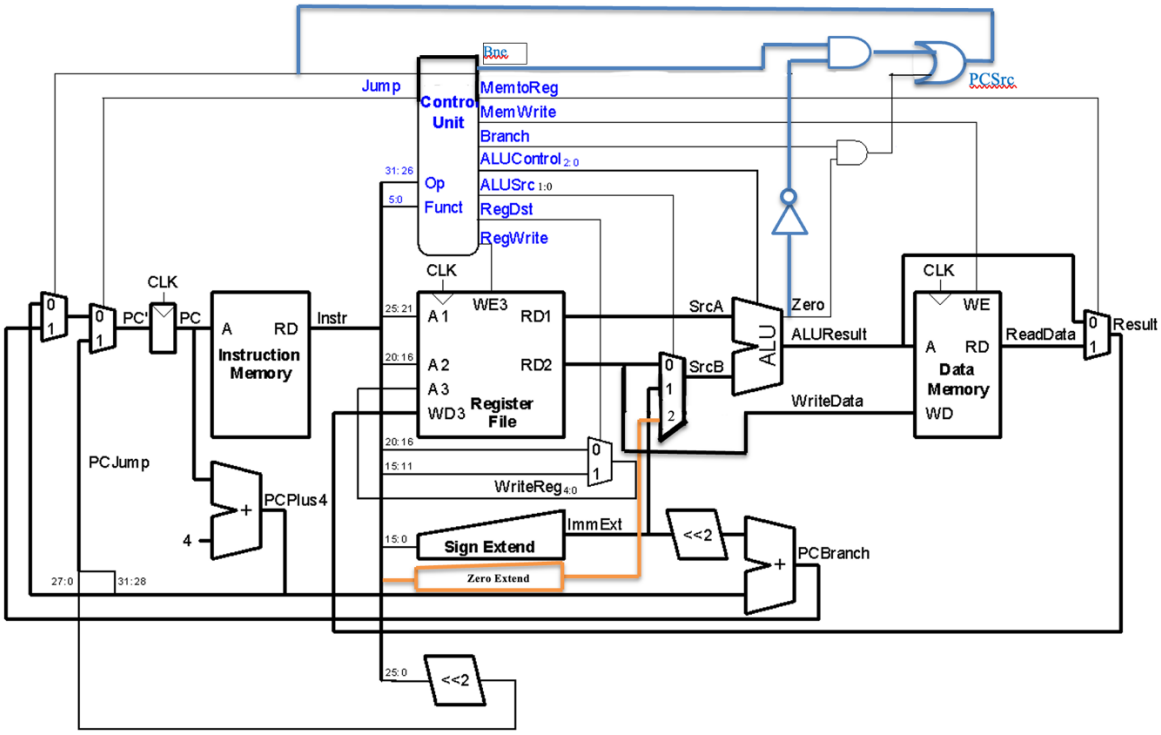








4. **Marked up versions of the datapath schematic and decoder tables that adds the** ori **and** bne **instructions.**



Single-cycle MIPS processor

5. **Your SystemVerilog code for your modified MIPS computer (including** ori **and** bne **functionality) with the changes highlighted and commented in the code and the contents of your** memfile2.dat **containing your test 2 machine language code:**

mips.sv

```
// single-cycle MIPS processor
module mips(input  logic        clk, reset,
            output logic [31:0] pc,
            input  logic [31:0] instr,
            output logic        memwrite,
            output logic [31:0] aluout, writedata,
            input  logic [31:0] readdata);

  logic        memtoreg, branch,pcsrc, zero,
               regdst, regwrite, jump;

  logic [1:0] alusrc
  // TODO: (1) Declare any new control signals needed for bne.
  // TODO: (2) alusrc is currently 1 bit. ori needs a third mux input
  //           (zero-extended immediate). Consider widening alusrc to [1:0].
  logic [2:0]  alucontrol;

  controller c(instr[31:26], instr[5:0], zero,
               memtoreg, memwrite, pcsrc,
               alusrc, regdst, regwrite, jump,
               alucontrol);
  datapath dp(clk, reset, memtoreg, pcsrc,
              alusrc, regdst, regwrite, jump,
              alucontrol,
              zero, pc, instr,
              aluout, writedata, readdata);
endmodule

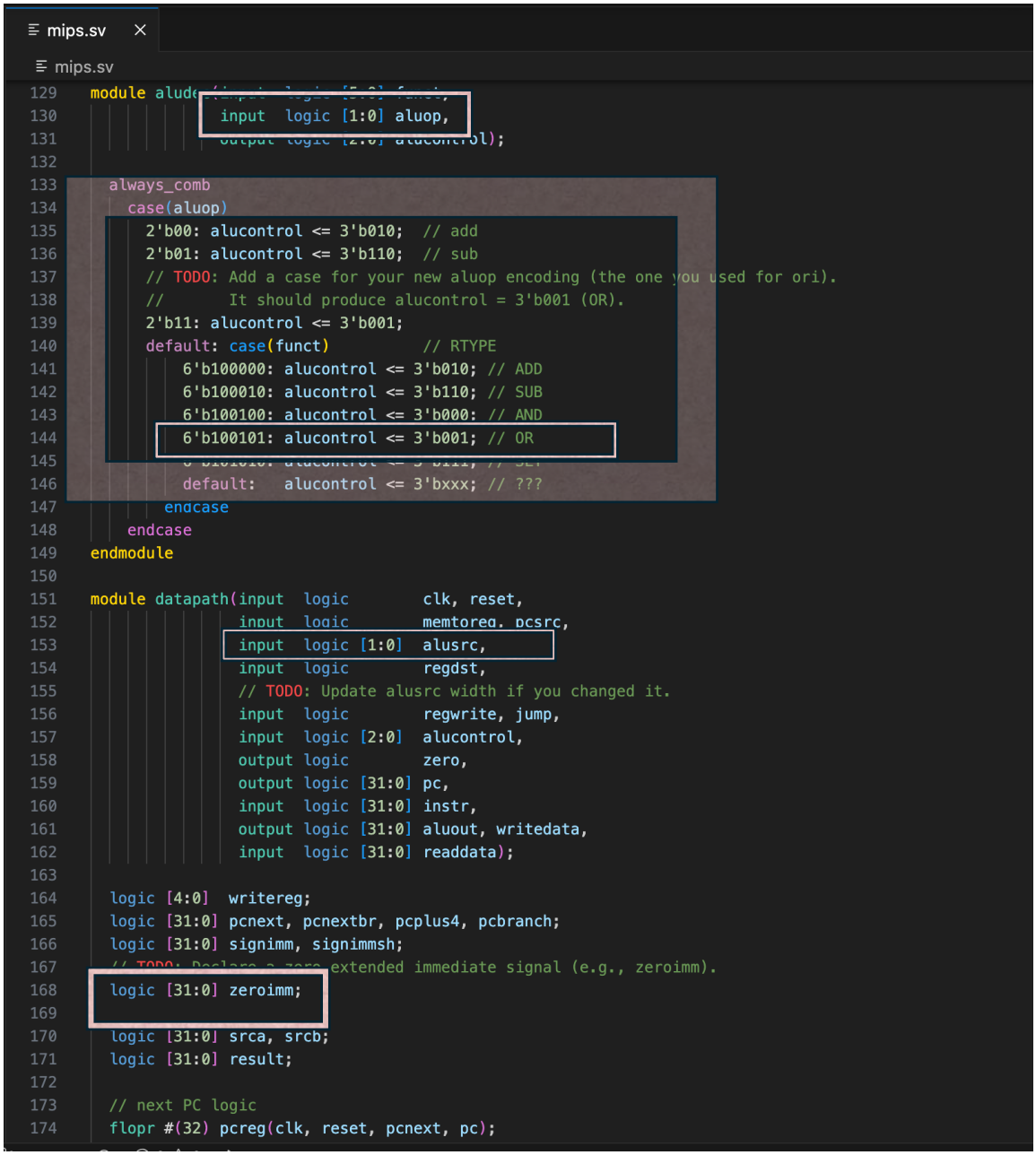
module controller(input  logic [5:0] op, funct,
                  input  logic       zero,
                  output logic       memtoreg, memwrite,
                  output logic       pcsrc,
                  output logic       [1:0] alusrc,
                  // TODO: Update alusrc width if you changed it above.
                  output logic       regdst, regwrite,
                  output logic       jump,
                  output logic [2:0] alucontrol);

  logic [1:0] aluop;
  logic       branch;
  // TODO: Declare a bne signal.
  logic bne;

  maindec md(op, memtoreg, memwrite, branch, bne, alusrc, regdst, regwrite, jump,aluop);
  // TODO: Pass the bne signal to/from maindec.
```

mips.sv

```
59  module controller(input  logic [5:0] op, funct,
80    // TODO: Modify pcsrc so that it is also asserted when bne is active
81    //       and zero is NOT set.  (beq branches when equal; bne branches
82    //       when not equal.)
83    assign pcsrc = (branch & zero) | (bne & ~zero);
84  endmodule
85
86  module maindec(input  logic [5:0] op,
87                 output logic       memtoreg, memwrite,
88                 output logic       branch, bne,
89                 // TODO: (1) Add a bne output port.
90                 // TODO: (2) Update alusrc width if needed.
91                 output logic [1:0] alusrc,
92                 output logic       regdst, regwrite,
93                 output logic       jump,
94                 output logic [1:0] aluop);
95
96    logic [10:0] controls;
97    // TODO: The controls vector must grow to hold the new fields (bne, wider alusrc).
98    //       Update its width accordingly.
99
100   assign {regwrite, regdst, alusrc,
101           branch, bne, memwrite,
102           memtoreg, jump, aluop} = controls;
103   // TODO: Add bne (and the extra alusrc bit) into this concatenation.
104   //       Make sure the bit positions match your new controls width.
105
106   always_comb
107     case(op)
108       6'b000000: controls <= 11'b11000000010; // R-type
109       6'b100011: controls <= 11'b10010001000; // LW
110       6'b101011: controls <= 11'b0x01001x000; // SW
111       6'b000100: controls <= 11'b0x00100x001; // BEQ
112       6'b001000: controls <= 11'b10010000000; // ADDI
113       6'b000010: controls <= 11'b0xxx000x1xx; // J
114       // TODO: Add a case for ori (opcode 6'b001101).
115       //       ori writes to a register (regwrite=1), uses rt as destination (regdst=0),
116       //       uses a zero-extended immediate (alusrc=??), does not branch or write memory,
117       //       and the ALU should perform OR. Pick a new aluop encoding (e.g., 2'b11).
118       //
119       6'b001101: controls <= 11'b10100000011; //ORI
120       // TODO: Add a case for bne (opcode 6'b000101).
121       //       bne is like beq but sets the bne signal instead of branch.
122       //       The ALU still subtracts to check equality (aluop=01).
123       6'b000101: controls <= 11'b0x00010x001; //BNE
124       default:   controls <= 11'bxxxxxxxxxxx; // ???
```


mips.sv
mips.sv
module alude
input logic [1:0] aluop,
output logic [2:0] alucontrol);
always_comb
case(aluop)
2'b00: alucontrol <= 3'b010; // add
2'b01: alucontrol <= 3'b110; // sub
// TODO: Add a case for your new aluop encoding (the one you used for ori).
// It should produce alucontrol = 3'b001 (OR).
2'b11: alucontrol <= 3'b001;
default: case(funct) // RTYPE
6'b100000: alucontrol <= 3'b010; // ADD
6'b100010: alucontrol <= 3'b110; // SUB
6'b100100: alucontrol <= 3'b000; // AND
6'b100101: alucontrol <= 3'b001; // OR
default: alucontrol <= 3'bxxx; // ???
endcase
endcase
endmodule
module datapath(input logic clk, reset,
input logic memtoreg, pcsrc,
input logic [1:0] alusrc,
input logic regdst,
// TODO: Update alusrc width if you changed it.
input logic regwrite, jump,
input logic [2:0] alucontrol,
output logic zero,
output logic [31:0] pc,
input logic [31:0] instr,
output logic [31:0] aluout, writedata,
input logic [31:0] readdata);
logic [4:0] writereg;
logic [31:0] pcnext, pcnextbr, pcplus4, pcbranch;
logic [31:0] signimm, signimmsh;
extended immediate signal (e.g., zeroimm).
logic [31:0] zeroimm;
logic [31:0] srca, srcb;
logic [31:0] result;
// next PC logic
flopr #(32) pcreg(clk, reset, pcnext, pc);

```
185   mux2 #(5)   wrmux(instr[20:16], instr[15:11],regdst, writereg);
186
187   mux2 #(32)  resmux(aluout, readdata,memtoreg, result);
188
189   signext     se(instr[15:0], signimm);
190
191   // TODO: Instantiate the zeroext module (defined in mipsparts.sv) to produce zeroimm.
192   zeroext     ze(instr[15:0], zeroimm);
193
194   mux3 #(32)  srcbmux(writedata, signimm, zeroimm, alusrc,srcb);
195   // TODO: The SrcB mux currently has 2 inputs (register vs. sign-extended imm).
196   //       You need a 3rd input (zero-extended imm) for ori.
197   //       Replace mux2 with mux3 (defined in mipsparts.sv) and connect
198   //       the three data inputs and the 2-bit alusrc select signal.
199   //       alusrc encoding: 00 = register, 01 = sign-ext imm, 10 = zero-ext imm
200   alu         alu(.a(srca), .b(srcb), .f(alucontrol),.y(aluout), .zero(zero));
201 endmodule
202
```

mipsparts.sv

```
52
53  // TODO: Add a zeroext module here.
54  //       It should have the same interface as signext (16-bit input, 32-bit output)
55  //       but pad the upper 16 bits with zeros instead of replicating the sign bit.
56  //       This is needed for the ori instruction, which uses zero-extended immediates.
57
58  module zeroext(input logic [15:0] a, output logic [31:0] y);
59    assign y = {16'b0, a};
60  endmodule
61
62  module flopr #(parameter WIDTH = 8)
63                (input  logic             clk, reset,
64                 input  logic [WIDTH-1:0] d,
65                 output logic [WIDTH-1:0] q);
66
67    always_ff @(posedge clk, posedge reset)
68      if (reset) q <= 0;
69      else       q <= d;
70  endmodule
71
72  module flopenr #(parameter WIDTH = 8)
73                  (input  logic             clk, reset,
74                   input  logic             en,
75                   input  logic [WIDTH-1:0] d,
76                   output logic [WIDTH-1:0] q);
77
78    always_ff @(posedge clk, posedge reset)
79      if      (reset) q <= 0;
80      else if (en)    q <= d;
81  endmodule
```

mipsparts.sv

```
72  module flopenr #(parameter WIDTH = 8)
78    always_ff @(posedge clk, posedge reset)
80      else if (en)     q <= d;
81  endmodule
82
83  module mux2 #(parameter WIDTH = 8)
84                (input  logic [WIDTH-1:0] d0, d1,
85                 input  logic             s,
86                 output logic [WIDTH-1:0] y);
87
88    assign y = s ? d1 : d0;
89  endmodule
90
91  // TODO: Add a mux3 module here.
92  //       It should be parameterized like mux2 (with a WIDTH parameter),
93  //       have three data inputs (d0, d1, d2), a 2-bit select input (s),
94  //       and one output (y).
95  //       Use a case statement inside always_comb:
96  //         s = 2'b00 -> y = d0
97  //         s = 2'b01 -> y = d1
98  //         s = 2'b10 -> y = d2
99  //       This mux is used in the datapath to select SrcB among:
100 //         register data (00), sign-extended imm (01), zero-extended imm (10).
101
102 module mux3 #(parameter WIDTH = 8)
103               (input  logic [WIDTH-1:0] d0, d1, d2,
104                input  logic [1:0]       s,
105                output logic [WIDTH-1:0] y);
106
107   always_comb
108     case(s)
109       2'b00: y = d0;
110       2'b01: y = d1;
111       2'b10: y = d2;
112       default: y = {WIDTH{1'bx}};
113     endcase
114 endmodule
```

memfile2.dat

```
1   34088000
2   20098000
3   350A8001
4   11090005
5   0128582A
6   15600001
7   08000009
8   01485022
9   350800FF
10  016A5820
11  01484022
12  AD680052
```

6. An image of the simulation waveforms showing correct operation of your modified processor on the new program.

In the new program (memfile2.dat), the sw instruction writes the data value 0xFFFF7F02 to the memory address 82 (0x52 in hexadecimal).

